

Design Pattern Analysis in a FastAPI-Based Logistics Management Application

Warehouse & Transportation Management System (WTMS)

1. Application Overview

This case study analyzes the Warehouse & Transportation Management System (WTMS), a FastAPI-based logistics management application using PostgreSQL. The application provides warehouse management, multi-warehouse inventory, storage zones and bins, stock movements, orders, vehicles, drivers, routes, shipments, shipment tracking, and transportation management. The baseline application uses a repository-based data-access architecture.

2. Environment

Python 3.11.5; PostgreSQL 18.6; OpenJDK/Temurin 21.0.1 LTS; Node.js 22.15.1; MinGW.org GCC 6.3.0 with C++17 and -O2.

3. Part 1: Baseline Pattern Analysis

The primary confirmed design pattern is the Repository Pattern. Four repository instances were confirmed from source-code evidence: WarehouseRepository, InventoryRepository, OrderRepository, and TransportationRepository. API endpoints obtain repository instances through FastAPI dependency injection, while repositories encapsulate database operations through the Database abstraction. FastAPI dependency injection and the Database abstraction were treated as supporting mechanisms rather than separate pattern instances. No additional pattern was counted without sufficient source-code evidence.

4. Part 2: Design Evolution

Multiple-warehouse support already exists in the baseline application and was verified with four warehouse records. Change 1 introduced delivery partners and shipment delivery_partner_id support. Change 2 introduced shipment tracking with latitude, longitude, location, status, and timestamps. Change 3 introduced shipment route history and a transactional route-change operation. Detailed evidence and UML diagrams are stored under CaseStudy/Part2.

5. Part 3: Cross-Language Implementation

The Repository Pattern was implemented independently in Java, Python, JavaScript, and C++. Each implementation saves a shipment, finds a shipment by ID, retrieves all shipments, and verifies the expected result. All four implementations passed their functional tests.

6. Measurement Methodology

For each implementation, compilation was performed where required, followed by one warm-up execution and five measured executions. PowerShell Measure-Command was used for process-level timing and the median of the five measured runs was recorded. The timings include process startup overhead and are specific to the local environment and demonstration programs.

7. Results

Java: 74 lines, cyclomatic complexity 3, median 81.214 ms, PASS. Python: 45 lines, complexity 1, median 93.486 ms, PASS. JavaScript: 49 lines, complexity 3, median 46.009 ms, PASS. C++: 65 lines, complexity 3, median 14.209 ms, PASS.

8. Evidence and Data

Pattern evidence is maintained in `patterns.csv` and `pattern_counts.csv`. Evolution is maintained in `patterns_evolution.csv`. Cross-language measurements are maintained in `cross_language.csv`. Part 1, Part 2, and Part 3 evidence files and UML diagrams are stored under `CaseStudy`.

9. Limitations

The cross-language programs are small representative implementations of the Repository Pattern and are not replacements for the complete WTMS backend. Process-level timing includes runtime startup overhead. The measurements should not be interpreted as general programming-language performance rankings. Patterns were counted only when supported by source-code evidence.

10. Conclusion

The baseline WTMS application demonstrates a Repository-based data-access architecture with four confirmed repository instances. The evolution scenarios show extensions for delivery partners, shipment tracking, and route-history management while preserving the repository abstraction. The cross-language implementations demonstrate the Repository Pattern in four languages with reproducible functional tests and measured process-level execution times.